

Archive Search — Technical Build Spec

Status: Draft technical spec — awaiting Peter's approval before Neo/Q build or run anything. Nothing in this document authorizes running the batch screening pass against real data. That is its own, separate, later gate — see "Before Any of This Runs For Real," below.

Written by: Oracle

Date: 2026-09-10

Origin: Translates `archive-search-v1-scope.md` (the finalized product design — Peter has resolved every open item in it, including the access model, corrected per Mason's real review) into a buildable engineering spec, and resolves every finding from Asimov's and Mason's real governance/legal review of that scope directly — not by reference, by design. Every product decision in the scope document is treated as settled and is encoded here faithfully, not re-litigated.

Built from, read in full:

- `projects/hub/email-intake/archive-search-v1-scope.md` — the source design this spec implements, including Section 4's resolved 8-person access model (7 real Missive-membership overlaps + Caylee as a named, accepted exception) and Section 2's Option A (one-time retroactive batch pass, two-layer Fair Housing check) recommendation.
- `GOVERNANCE.md` and `CLAUDE.md` — the compliance-build trigger, the ten Rules, the Fair Housing Standard.
- `supabase/migrations/20260905020000_missive_shared_inbox_intake_schema.sql` (full) — the real, live `missive_message_intake/missive_sync_state` schema, its Rule 9 housing-decision firewall, and — load-bearing for this spec specifically — its own `retention_policy` note naming the exact PREREQUISITE this spec's new column satisfies (quoted directly below).
- `supabase/migrations/20260815010000_maintenance_history_schema.sql` — the real `maintenance_claims_decision_safe` view, the pattern this spec's own `missive_message_intake_search_safe` view mirrors exactly (single-table WHERE filter, no join, read by convention never by enforcement).
- `projects/hub/email-intake/complaint-tracking-technical-spec.md` and its real, now-built implementation (`projects/hub/complaint-tracking/router.js`, `lib/process-pending-messages.js`, `lib/thread-adapter.js`) and `compliance/complaint-tracking-ai-risk-assessment.md` — the structural and rigor template for this document, and the real, live pipeline this spec must not collide with (see "Resolving Finding 2" and "Resolving Finding 9," below — this spec reuses `thread-adapter.js`'s real, exported functions rather than re-deriving them).
- `projects/hub/email-intake/lib/privilege-filter.js`, `privilege-keywords.js` — the real, live hold check, and the real gaps Mason found in it (Finding 4, below).
- `projects/hub/maintenance-history/lib/content-check.js`, `protected-class-terms.js`, `tier-b-classifier.js` — the real, live two-layer Fair Housing content check this spec reuses unchanged, and the fail-closed AI-classifier pattern this spec's own new classifier is modeled on directly.

- `projects/hub/owner-tenant-notes/router.js` — the real `access_tier/placeholder` visibility pattern, read specifically to confirm it does **not** apply here (see "Resolving Finding 6," below).
- `supabase/migrations/20260815000000_audit_log_rule1_compliance.sql`, `20260910000000_complaint_tracking_schema.sql` — `audit_log`'s real, current CHECK constraints and the real, current `team_member_tool_roles` tool/role lists (confirmed live: 11 tool values as of `complaint_tracking`, 9 role values as of `maintenance_coordinator` — this migration is the 12th and 10th respectively).
- `compliance/complaint-tracking-ai-risk-assessment.md` — the structure and tone this spec's own companion document, `compliance/archive-search-ai-risk-assessment.md`, is modeled on.

Where this will live: `projects/hub/archive-search/` (`router.js`, `lib/`, `dashboard/index.html`) — a new Hub section, mounted into `projects/hub/server.js`, gated by `team_member_tool_roles` with a new tool value `'archive_search'`. Same shape every other Hub tool uses. This pass ships the schema and the API only (Tron builds `dashboard/index.html` separately, same division of labor the `complaint-tracking` build just used).

Neo's Schema Review — 2026-09-10

Verdict up front: sound to build from, after three fixes made directly in this document below — not sound as originally drafted. Read this spec in full against the real prior art and real live code it cites:

`supabase/migrations/20260905020000_missive_shared_inbox_intake_schema.sql` (full — RULE 1/RULE 2/PREREQUISITE language, RLS notes, Rule 9 firewall),
`supabase/migrations/20260815010000_maintenance_history_schema.sql` (the real `maintenance_claims_decision_safe` view this spec's own view claims to mirror),
`supabase/migrations/20260910000000_complaint_tracking_schema.sql` and
`20260902020000_add_maintenance_coordinator_role.sql` (the real, current `team_member_tool_roles` CHECK lists — confirmed the spec's "11 tool values, 9 role values" count is exactly right, not assumed), `supabase/migrations/20260815000000_audit_log_rule1_compliance.sql` (the real `audit_log` CHECK constraints — every `actor_type/privacy_category/risk_level` value this spec uses is confirmed legal today),
`projects/hub/complaint-tracking/lib/process-pending-messages.js` (confirmed, by reading the real code, that its ingestion query really is `WHERE pipeline_status = 'pending'` with no other filter — the Finding 2 sequencing claim is accurate, not assumed),
`projects/hub/complaint-tracking/lib/thread-adapter.js` (confirmed `toThreadShape/threadFullText` are real, exported functions, not invented signatures), and
`projects/hub/email-intake/lib/privilege-keywords.js` /
`projects/hub/maintenance-history/lib/protected-class-terms.js` (confirmed, by reading the real files, that `'lawyer'` really is missing from `HOLD_TERMS`, the Fair Housing complaint phrases really are exact-phrase-only, and no standalone `discriminat*` trigger exists today — Finding 4's diagnosis is accurate, and the version bumps `v3→v4` / `v1→v2` are the correct next numbers). Three things did not check out as originally drafted, and are fixed in place below rather than just described:

1. **The Finding-1 enforcement gate is real but weaker than the surrounding language implies, and the risk is higher here than for the view it mirrors.** The spec is honest that the gate is "enforced by code convention... not by a database mechanism" — that part is not a misrepresentation. What the original draft

missed: `maintenance_claims_decision_safe` (the pattern being mirrored) is a plain view over already-AI-screened output; this new view sits over `missive_message_intake`, this schema's own highest-PII-density table, storing content *before* any privilege/Fair-Housing check runs — and the new `search_document` generated column is computed over **every** row, held and flagged ones included, so the GIN index itself physically contains tokenized privileged content. A query that accidentally hit the base table instead of the view wouldn't just leak ordinary mail — it could return privileged/Fair-Housing-flagged correspondence in a search result. Fixed below: `security_barrier = true` added to both `CREATE VIEW` statements (a real Postgres mechanism, not decorative — see "Resolving Finding 1"), plus an explicit, honest statement of what that option does and does not cover, plus a concrete, cheap code-level guardrail recommendation for the part it doesn't cover.

2. Finding 9's retention consequence was described incompletely. The original text states the 4-year clock starts on every 'clear' row and that held rows get a permanent hold — but says nothing about 'flagged_protected_class' rows. Read against the migration's own RULE 2 language (a strict binary: Tier 2 HELD, or "everything else"), and against this codebase's own established precedent that a Fair-Housing flag is "advisory only, never a hold" (`complaints.flagged_protected_class`'s real column comment, `20260910000000_complaint_tracking_schema.sql`), a flagged-but-not-held row is **not** held — it falls under "everything else" and gets the identical ordinary 4-year clock a 'clear' row gets, even though it's excluded from search. Left unstated, this reads as if "flagged" might mean "protected like held" — it doesn't. Fixed below in both "Resolving Finding 9" and "Open Items."

3. A real migration-execution risk on a 254,056-row production table, not called out anywhere in the original draft. `ADD COLUMN search_document tsvector GENERATED ALWAYS AS (...) STORED` is not a metadata-only change in Postgres the way a plain nullable column add is — it requires a full table rewrite to compute the expression for every existing row, holding an exclusive lock on `missive_message_intake` for the duration, while the live Missive sync cron job (which writes to this exact table) keeps running on its own schedule. The plain `CREATE INDEX ... USING GIN` in the same block has the same non-concurrent lock problem. This project has no staging copy of the data (the same standing caveat the original `20260905020000` migration's own gate self-check already carries) — which makes getting the real table's lock behavior right the first time, on the real table, more important, not less. Fixed below: the new indexes are built `CONCURRENTLY`, called out as statements that must run on their own, outside any wrapping transaction, with an explicit operational note for Peter about what to expect.

Everything else — the `screening_result/screening_category/screening_tags/screening_version/screening_completed_at` column set, the CHECK constraints' logic (both are consistent with this table's real, current constraints and with the `complaints_flag_requires_category` pattern they're modeled on), the `team_member_tool_roles` CHECK additions (both counts confirmed exactly right against the real, live lists), every `audit_log` event's field values (confirmed legal against the real CHECK constraints), the Finding 2 self-report design and its fail-closed behavior, the Finding 4 keyword/co-occurrence fixes, the Finding 5 validation-sample design, and the Finding 8 held-export design — is consistent with the rest of this schema and does not duplicate anything another table already owns. Detail on each fix is inline at its original location below, not repeated a third time here.

What This Does

Every message ever received into Rincon's two Missive Team Inboxes (254,056 of them, verbatim, back to November 2022 and a handful of older stray messages) already sits in Rincon's own database, unsearchable.

This build makes it searchable — type a name, an address, or a phrase, get back every message that mentions it, newest first — for the 8 people who already have Property 360 access and (for 7 of them) already have hand-access to this same correspondence in Missive today. Before any of that can go live, a one-time pass runs the same two safety checks this codebase already uses elsewhere (the privilege/legal-hold check, the two-layer Fair Housing content check) against all 254,056 messages, so a held or Fair-Housing-flagged thread can never appear in a search result — not because the search code remembers to filter it out, but because it was never in the pool search is allowed to read from.

How It Works

1. **The screening pass** (POST /api/archive-search/process-pending, manually triggered, x-cron-secret-gated) reads every `missive_message_intake` row not yet screened by this tool, groups it into conversations, and runs `checkThread()` (the real hold check) on each one.
2. **Held** conversations (formal legal/Fair-Housing-complaint signals) are recorded as `screening_result = 'held'` and go no further — never read for content-check purposes, never searchable.
3. **Everything else** gets the real two-layer Fair Housing content check (`checkClaim()`), with a new, narrow, single-purpose AI self-report call standing in for the full categorization call complaint-tracking's own pipeline uses for Layer 2 (see "Resolving Finding 2"). A flagged thread is recorded `screening_result = 'flagged_protected_class'` and is also excluded from search. Everything else is recorded `screening_result = 'clear'`.
4. **Search only ever reads a Postgres view** (`missive_message_intake_search_safe`) that structurally contains `screening_result = 'clear'` rows and nothing else — the same `maintenance_claims_decision_safe` pattern already proven in this codebase.
5. **A person searches** (GET /api/archive-search?q=...), gets back sender/date/subject/snippet, newest first, with a link to the real Missive conversation. Opening a message logs who looked at what.
6. **New mail** keeps landing in `missive_message_intake` automatically (the existing Missive connector); this tool's own screening pass, re-run periodically by hand, is what keeps the searchable pool current — deliberately decoupled from whether or when the complaint tracker's own pipeline runs (see "Resolving Finding 2 / Finding 9" for exactly how).

What You'll See

Exactly per the product doc's Section 1: a search box, a list of matches (sender, date, a short snippet with the matched text in context, newest first), click one for the full message with a link back to the real Missive thread. No AI summary, no "here's what this means" — a fast, reliable find, nothing more, for v1.

What Could Go Wrong

- **The screening pass has a real, one-time cost in both time and money** — a narrow AI call for every non-held conversation across a quarter-million messages is not free or instant; see "Build Size and Runtime" for how this spec asks Q to run it in resumable chunks, not one unattended pass.

- **A false negative on either check is the single highest-stakes failure mode** — see the companion risk assessment for how this is mitigated (keyword-gap fixes, the validation sample, the fail-closed classifier) and what residual risk remains, honestly stated.
- **This is the first tool in the Hub that gives 8 people day-to-day access to raw, unscreened-by-a-human correspondence at this scale** — the search-activity log (Finding 6, below) is the real, deliberate mitigation for misuse, not an afterthought.

Resolving the Real Asimov + Mason Review

The scope document names nine real findings from Asimov's and Mason's review of the product design, without yet giving each one a concrete engineering answer. This section is that answer, one finding at a time. Nothing here re-opens a product decision the scope document already settled (access population, batch-vs-live screening, which content check to reuse) — this is translation into buildable design, not a second round of the product conversation.

Finding 1 — A real, enforced technical gate

The gap: `missive_message_intake.pipeline_status` only ever recorded *that* the Stage 0/1/2 filter ran, never *what it found*. Nothing stops a future query from reading the base table directly and returning a held or flagged thread.

The fix — a new column, not a new table. `missive_message_intake` gains:

```
```sql
```

```
screening_result TEXT -- 'held' | 'flagged_protected_class' | 'clear' | NULL (not yet screened)
```

```
screening_category TEXT -- populated only when screening_result = 'flagged_protected_class'; mirrors
complaints.flagged_category
```

```
screening_tags JSONB -- Tier 1 TAG labels from checkThread() (e.g. ['regulatory_matter']) — informational, not a
search filter, kept for the held-review export and any future UI
```

```
screening_version TEXT -- which version of the privilege/content-check terms + self-report classifier produced this
result — see "Versioning" below
```

```
screening_completed_at TIMESTAMPTZ -- when this row was actually screened, distinct from
missive_message_intake's own deliberate lack of a generic updated_at
```

```
```
```

A new table was considered and rejected: it would need its own 1:1 join back to `missive_message_intake` for the safe view to work at all, adding a join where the existing `maintenance_claims_decision_safe` precedent has none, for no real benefit — and the original migration's own `retention_policy` note (quoted next) already names a same-table column as the expected shape.

Why this column, specifically, is not just a Finding-1 fix. The original migration states its own real limitation directly: *"Before any age-based deletion job is ever built against this table, it must be able to answer, per row, 'was this Tier 2 HELD' — either via a new column on this table (a held/tier flag; Neo's call, not added by this migration) or a reliable join to wherever the filter actually records its output. Until that mechanism exists, RULE 1 continues to apply even to 'processed' rows."* `screening_result` is that column — this spec is not just satisfying Asimov's Finding 1, it is satisfying a real, already-named prerequisite in the live schema that nothing has built yet. See "Resolving Finding 9" for the retention consequence this actually triggers.

The view — mirroring `maintenance_claims_decision_safe`'s single-table WHERE, no join, no exception logic — plus one real addition `maintenance_claims_decision_safe` doesn't need (see Neo's review, fix 1):

```
```sql
CREATE OR REPLACE VIEW missive_message_intake_search_safe
WITH (security_barrier = true) AS
SELECT *
FROM missive_message_intake
WHERE screening_result = 'clear';
```
```

Same RLS caveat as the maintenance schema's own note on that view (Postgres views run as their owner, not the querying role — a non-issue here because every reader in this codebase, and this tool, connects via the Supabase service-role key, which bypasses RLS regardless). **Every route in `archive-search/router.js` and `archive-search/lib/` queries this view, never `missive_message_intake` directly — enforced by code convention (the same discipline `maintenance_claims_decision_safe`'s own header states), not by a database mechanism, since the service-role key that every Hub route uses bypasses RLS regardless of what the base table's policies say.** The one narrow exception is the screening pass itself (`lib/screening-pass.js`), which by definition must read the base table to screen it — and never selects `body_html/body_text` back out for display, only to feed the checks.

Why `security_barrier = true`, when `maintenance_claims_decision_safe` doesn't have it. `maintenance_claims_decision_safe` sits over already-AI-screened output; this view sits over this schema's own highest-PII-density table, holding content *before* any privilege/Fair-Housing check runs — and the `search_document` generated column below is computed over **every** row, held and flagged ones included, so the underlying GIN index physically contains tokenized privileged/Fair-Housing content even though the view is the only thing standing between a query and it. `security_barrier` is Postgres's own documented mechanism for exactly this case — a view meant to enforce row visibility, not just convenience — and it costs nothing here: both predicates this view's queries actually evaluate (`screening_result = 'clear'` and, per "The Search Mechanism" below, `search_document @@ websearch_to_tsquery(...)`) use built-in, leakproof operators (`=` and `tsvector's @@`), so Postgres's planner can still push them down and use the GIN index — the "no second index is needed on the view itself" and "still uses the GIN index" claims below hold exactly the same with this option added.

The real, honest limit that remains — stated plainly, not glossed over. `security_barrier` stops a leaky-function side channel; it does **not** stop a route in `archive-search/router.js` or `lib/` from simply querying `missive_message_intake` by name instead of the view — that boundary is still code discipline, not the database, because the one shared Supabase service-role key every Hub route already uses has full table access regardless of any view or RLS policy. A genuinely database-enforced boundary would mean giving `archive-search`'s read routes their own, separately-scoped Postgres role with `SELECT` revoked on `missive_message_intake` and granted only on the view — a real option, but a change to how every Hub tool connects to Supabase today (all of them share one service-role connection), and out of scope for a one-off fix on a single tool. **Recommended instead, as the practical backstop given that shared architecture: a small, cheap automated guardrail** — a test or CI check asserting that no file under `archive-search/router.js` or `archive-search/lib/`, other than `screening-pass.js`, contains the literal string `missive_message_intake` outside of `missive_message_intake_search_safe`. Cheap, catches the mistake at review/CI time rather than relying on every future diff being read carefully by hand. This is Q's build item, named here so it isn't dropped. Flagged as a real, accepted residual risk, not hidden: even with this guardrail, the boundary is convention plus a lint check, not a database guarantee — for the reason stated above.

The view's `WHERE screening_result = 'clear'` predicate composes correctly with a full-text search predicate on the same table (Postgres flattens a `security_barrier` view's leakproof predicates into the outer query the same way it does for a plain view, so the GIN index described in "The Search Mechanism" below is still used for a query against the view, not just the base table) — confirmed so Q doesn't duplicate the index anywhere.

A `CHECK` constraint enforces the same discipline `complaints_flag_requires_category` already does on `complaints`:

```
```sql
ALTER TABLE missive_message_intake
ADD CONSTRAINT missive_message_intake_screening_category_required
CHECK (screening_result IS DISTINCT FROM 'flagged_protected_class' OR screening_category IS NOT NULL);
```
```

What is deliberately *not* logged onto this row, matching Design Decision 2 of the complaint-tracking spec exactly: the matched keyword term, the raw Tier B classifier output, or the model's own self-report text. Only `screening_category` (the category label, e.g. `disability_health`) and `screening_tags` (Tier 1 labels only, never Tier 2 hold reasons) live on the row. Matched-term detail, when it exists at all, goes only into `audit_log.details` for the held/flagged audit events (Finding 6) — and this spec does **not** extend `protected-class-terms.js`'s one narrow, already-approved Tier B logging exception (scoped by name to `maintenance_claims.tier_b_classification`) to this new tool, without Asimov's separate say-so.

Finding 2 — The real "1.5 layers, not 2" gap, and the real sequencing answer

The gap, confirmed against the real, now-built code, not hypothetically. `checkClaim()`'s Layer 2 is not a second, independent check — it's a self-report from whatever model is *already* reading the full text for another purpose. Complaint-tracking's own pipeline gets this for free because its categorization step already reads every thread. Archive search's batch pass has no other reason to read a thread's content at all — if it only calls

checkClaim({ claim_text, modelFlag: undefined, modelCategory: undefined }), modelFlag is always falsy, layer2Hit is always false (per content-check.js's own checkClaim(): const layer2Hit = !!claim.modelFlag), and the pass is really running Layer 1 (Tier A + Tier B) only, dressed up as "the two-layer check."

Two real options, and a real recommendation.

- **Option A — be honest that it's Tier A/B only.** Cheaper, faster, zero new AI-call surface. But this directly undercuts the product doc's own stated reasoning for choosing the advanced two-layer check over the simpler keyword-only fair-housing-filter.js in the first place: *"the more advanced, two-layer version... already been precision-tuned with Mason and outside counsel"* — running only its Layer 1 half on a one-time pass whose result gets trusted for years quietly reverts to the simpler check the product doc explicitly rejected.

- **Option B — a narrow, single-purpose self-report call, not a full categorization call.** New module, archive-search/lib/fair-housing-batch-self-report.js, modeled directly on tier-b-classifier.js's real, proven shape: one Claude call per non-held conversation, claude-sonnet-5, max_tokens: 512, output_config: { effort: 'low' }, a 12000ms timeout (same precedent range as TIER_B_TIMEOUT_MS), asking exactly one narrow question — does this thread describe or reference a protected characteristic that an automated keyword scan might not already catch — and returning a small JSON object, never free text. **Fails closed on every error path, identically to tier-b-classifier.js:** a timeout, a network error, or an unparseable response resolves to { flagged: true, category: 'model_self_report_failed_closed' }, never a thrown exception, never a silent pass.

Recommendation: Option B. The product doc's own Option A reasoning for the batch pass itself — *"the cost is the same work either way — the only question is whether you pay it once or over and over"* — applies identically one level down, to Layer 2 specifically: a narrow, cheap-per-call classifier paid once across a bounded, known conversation count is a real but bounded cost, and Layer 2 exists specifically to catch the subtler phrasing Layer 1's keyword list structurally cannot — which Finding 4 (below) independently proves has real, live gaps today. Settling for Tier A/B only on a pass whose output gets trusted indefinitely is the wrong place to cut the one corner the product doc already argued against cutting.

```
``js
```

```
// archive-search/lib/fair-housing-batch-self-report.js
```

```
const FAIR_HOUSING_SELF_REPORT_VERSION = 'archive-search-self-report-v1';
```

```
async function selfReportFairHousingContent({ threadText }) {
```

```
// Same client()/timeout/fail-closed shape as tier-b-classifier.js's
```

```
// classifyTierBTerm() — see that file for the precedent this mirrors.
```

```
// Prompt asks for exactly one JSON object: { flagged: boolean, category: string|null }.
```

```
// Any error, timeout, or unparseable response resolves to
```

```
// { flagged: true, category: 'model_self_report_failed_closed' } — never thrown.
```

```
}
```

```
module.exports = { selfReportFairHousingContent, FAIR_HOUSING_SELF_REPORT_VERSION };
...

```

Call shape into the existing, unchanged `checkClaim()`:

```
```js
const selfReport = await selfReportFairHousingContent({ threadText });
const contentCheck = await checkClaim({
 claim_text: threadText,
 modelFlag: selfReport.flagged,
 modelCategory: selfReport.category,
});
...

```

**The real sequencing answer for Section 3's open question.** The scope document names a real risk: if the complaint tracker's own ingestion pipeline (`complaint-tracking/lib/process-pending-messages.js`) is ever pointed at the full 254,056-message backlog, it does not stop at the hold check — every non-held thread goes straight into full six-category AI categorization, which `complaint-tracking-ai-risk-assessment.md` requires Peter and the DO to review **every single one** of, for 14 days — a commitment built for a normal day's trickle of new mail, not a quarter-million historical messages landing in one run. Confirmed directly against the real, live code (`process-pending-messages.js`'s `runProcessPendingMessages()`): its very first step is `SELECT * FROM missive_message_intake WHERE pipeline_status = 'pending'` — it does not distinguish old mail from new, at all.

**This spec's screening pass resolves that risk structurally, as a real, deliberate side effect, not an accident.** Per "Resolving Finding 9," this pass also sets `pipeline_status = 'processed'` for every row it screens — which is the same column complaint-tracking's own pipeline checks. Once this pass has run against the historical backlog, complaint-tracking's own `WHERE pipeline_status = 'pending'` query will no longer see any of it — meaning the exact risk the scope document names (complaint-tracking accidentally hoovering up a quarter-million messages into full categorization) becomes structurally impossible, not just unlikely, the moment this pass completes. **This is a real, permanent consequence, not an incidental one — flagged explicitly for Peter's confirmation in "Open Items," not assumed.**

The two pipelines can otherwise run fully independently and in parallel, per the scope document's own framing — they share exactly two functions (`checkThread`, `checkClaim`) and zero tables. Archive search's own screening-pass work queue is `WHERE screening_result IS NULL`, entirely decoupled from `pipeline_status` — so even a row complaint-tracking has already marked 'processed' (new mail it got to first) is correctly re-screened by this pass if `screening_result` is still `NULL`, and vice versa: this pass never skips a row just because complaint-tracking already touched it, and never re-runs complaint-tracking's own categorization.

## Finding 3 — The AI Risk Assessment

Written as its own companion document, modeled directly on `compliance/complaint-tracking-ai-risk-assessment.md`'s real structure: `compliance/archive-search-ai-risk-assessment.md`. See that document in full. It is required to exist and be reviewed before the batch pass runs against real data — this technical spec does not authorize that run.

## Finding 4 — The real keyword-list gaps

Both fixes below are real, scoped edits to real, existing files — not a rewrite of either.

`privilege-keywords.js` — **two fixes, version bumped to `privilege-keywords-v4`:**

1. Add 'lawyer' to `HOLD_TERMS`, alongside the existing 'attorney'. One-line addition, same `\b`-bounded compilation every other plain-word term already gets.

2. **Word-order tolerance for the Fair Housing complaint phrases.** Today 'fair housing complaint', 'hud complaint', and 'crd complaint' are exact-phrase matches — "a complaint about fair housing" or "filed a complaint with HUD" does not match any of them, and no amount of adding more literal phrases closes that gap in general (word order has too many real permutations to enumerate). The real fix is a new match type, independent of the flat term list, checked in addition to it:

```
```js
// New in privilege-keywords.js — a co-occurrence check, not a phrase list.
// Fires HOLD when a regulator/subject-matter token and a complaint-word
// both appear anywhere in the same text, regardless of order or distance
// — two independent regex tests, ANDed, not one .*-spanning regex (which
// would risk catastrophic backtracking over a full message body).
const HOLD_COOCCURRENCE_PAIRS = [
{
a: /\b(fair housing|housing discrimination|hud|department of housing and urban development|crd|civil rights
department|dfeh)\b/i,
b: /\bcomplaint(s|ed|ing)?\b/i,
},
];

// scanForHoldKeywords(text) is extended to also run each pair — a hit adds
// a synthetic entry (e.g. 'fair_housing_complaint_cooccurrence') to
// matchedTerms alongside any literal-phrase matches, so callers (and
```

```
// audit_log) can tell which mechanism fired. checkThread()/checkMessage()

// in privilege-filter.js need NO changes — they only ever call

// scanForHoldKeywords(text) and read .matched/.matchedTerms.

...


```

This is a real, scoped change contained entirely inside `privilege-keywords.js`; `privilege-filter.js` is untouched.

`protected-class-terms.js` — **one fix, version bumped to** `protected-class-terms-v2`: no standalone `discriminat*` trigger exists anywhere in the file today — only the phrase 'age discrimination' under the age category. Add a new category using the file's own existing, unchanged compilation mechanism (every term already gets a `\b`-bounded regex automatically via `FLAT_TERMS` — no function code changes needed, only a new data entry):

```
```js
discrimination_general: [
'discriminate', 'discriminated', 'discriminating', 'discriminates', 'discrimination', 'discriminatory',
],
...

```

This is squarely consistent with the file's own stated design stance, quoted directly: *"this list is intentionally broad/recall-oriented, not precision-tuned... A false NEGATIVE is the failure mode Rule 9 exists to prevent. When in doubt, this list errs toward flagging."* An accusation of discrimination in property-management correspondence is, in practice, essentially never an unrelated use of the word — this is exactly the kind of addition that list's own design philosophy calls for.

## Finding 5 — The validation-sample design

**Population:** every row with `screening_result = 'clear'` after the batch pass — the "clear" pool specifically, per Asimov's Finding 4 / Mason's Condition 1, not the flagged or held pool (a false positive there just costs an unnecessary human look; a false negative in the clear pool is the one that actually reaches a searcher).

### Stratified, oversampled from the pre-2024 era:

- Stratum A — `delivered_at >= '2024-01-01'`: **500 messages**, randomly sampled.
- Stratum B — `delivered_at < '2024-01-01'` (November 2022 through end of 2023, plus the ~61 pre-November-2022 stray messages): **500 messages**, randomly sampled.

Stratum B is a materially smaller share of the archive's total volume than Stratum A (the mailbox's message volume has grown over time), so an equal 500/500 split is a deliberate oversample of the earlier era, not a proportional one — directly per Asimov's Finding 4. **Total: 1,000 messages.**

### Mechanism: a plain exported list, not a review-queue UI. GET

/api/archive-search/validation-sample-export (admin-only), returning the 1,000-row stratified sample as CSV: message id, conversation id, delivered\_at, from\_address, subject, body\_text (the reviewer needs the real content to judge a miss — this is the one route in this tool that deliberately exposes screened 'clear' content in bulk to a human reviewer, which is the entire point of the exercise), and a Missive deep link. Reasoning for a plain export over a built review-queue feature: this is a one-time, bounded task, not an ongoing workflow — building persistent review-queue UI (state per row, reviewer assignment, a resolution flow) is real Tron/Q work that a one-time 1,000-row spot-check doesn't need, matching CLAUDE.md's own "keep everything as simple as possible" instruction. The trade-off, stated plainly: no structured, queryable record of which rows were reviewed or what each reviewer decided — mitigated by logging the review's own completion as a single audit\_log event (below), with written findings kept as a plain note (the same convention this codebase already uses for review documents, e.g. this spec itself, rather than a database table).

**Exit rule: zero confirmed misses.** A "miss" is a message the batch pass marked 'clear' that a human reviewer determines should have been 'held' or 'flagged\_protected\_class'. If the 1,000-row sample comes back with **any** confirmed miss: the tool does not launch. The root cause is investigated (a keyword-list gap not yet caught, a self-report classifier failure, a genuinely novel pattern), the fix is made, the **entire batch pass is re-run** (not just the sample), and a fresh sample is drawn before this rule is checked again. This mirrors, word for word in spirit, the same "zero confirmed misses on privilege/Fair-Housing content" bar the upstream privilege filter's own exit criteria already use, and the exact bar complaint-tracking-ai-risk-assessment.md sets for its own upstream reuse of these same checks.

**Who reviews it, and when:** per the scope document's own still-open sequencing note (Section 4 — "initial access... start narrower — limited to whoever does the pre-launch accuracy validation — and widen to the full 8 only once that sample comes back clean"), this sample is reviewed by whoever holds the 'admin' role for tool = 'archive\_search' (Peter, and whoever else Peter designates) — **before** any of the other 7 confirmed users are granted the 'searcher' role. No schema mechanism is needed to enforce this sequencing; it's operational (Peter simply doesn't insert the other team\_member\_tool\_roles rows until the sample has passed) — stated here so it's a deliberate step, not an assumption.

archive\_search.validation\_sample\_reviewed(audit\_log, actor\_type: 'human', risk\_level: 'low', privacy\_category: 'processing') is written once, with details: { sample\_size: 1000, misses\_found: <n>, reviewer, stratum\_a\_count: 500, stratum\_b\_count: 500 }.

## Finding 6 — Search-activity logging

**Required, reused exactly, audit\_log, no new table.** Two events, at the two real privacy-sensitive moments a searcher actually touches this archive:

Event	action	actor_type	entity_type	privacy_category	risk_level
-------	--------	------------	-------------	------------------	------------

---	---	---	---	---	---
-----	-----	-----	-----	-----	-----

A search is run	archive_search.query_performed	human	archive_search_query	processing	medium
-----------------	--------------------------------	-------	----------------------	------------	--------

A specific message is opened	archive_search.message_opened	human	missive_message_intake	processing	medium
------------------------------	-------------------------------	-------	------------------------	------------	--------

`risk_level: 'medium'` (not `'low'`, the usual default for a routine read) is a deliberate choice, directly reflecting the scope document's own words about this tool: *"a materially wider window into raw correspondence than anything else in the Hub today."*

`details for query_performed` includes the literal query text `{ query_text, result_count }`. This is a real, deliberate call, not an oversight — stated here so it's confirmed, not assumed: the actual audit value of "who searched what" (catching a staff member searching something they have no legitimate reason to look up — an ex-partner's name, a competitor, idle curiosity about a specific tenant) depends entirely on knowing *what* was searched, not just that a search happened. This is different from the "never log the matched term" rule that governs the Fair Housing content-check flags (Finding 1) — that rule exists to avoid creating a second, unredacted copy of privileged/flagged *content* elsewhere in the system; a search query a person typed is not archive content, it's an act, and logging it is the entire mechanism by which this tool's own use gets audited. **Precisely because this log is itself sensitive** (it is arguably the single most revealing log in the Hub about staff behavior), it is readable only by `'admin'` role holders for `tool = 'archive_search'` — a narrower population than the 8 people who can run searches — never surfaced to the searchers themselves, and never exported alongside message content.

**What is deliberately *not* computed:** how many additional (held/flagged) rows a query *would* have matched, had they not been excluded. Computing that number requires running the same search against the unscreened base table — exactly the code path Finding 1 exists to make structurally impossible. This spec does not add it back in for the sake of a more informative audit log.

## Finding 7 — The "no results" default, and why it's not the `operational_notes` placeholder pattern

**Confirmed as the safe default, designed concretely: a held or flagged message is structurally absent from every query this tool ever runs — not filtered at render time, not shown as a placeholder.** Because every search route reads exclusively from `missive_message_intake_search_safe` (Finding 1), a held or flagged thread cannot appear in a result set, a count, or any metadata field — there is no code path that has to remember to hide it, because the row was never fetched in the first place.

**Why this is genuinely not the same situation as `operational_notes`' own restricted-content placeholder, stated plainly rather than assumed:** `operational_notes` shows a placeholder for a note a viewer knows exists but can't see the full content of, because that tool is a **curated, purposeful record** — a property manager benefits from knowing "there's a restricted note here, go ask someone," because the whole point of that system is routing sensitive-but-relevant operational facts to the right person. Archive search is the opposite case: it surfaces **raw, unfiltered personal correspondence**, where the mere fact that a matching thread exists is itself sensitive information. A searcher typing a tenant's name and seeing "1 result, restricted" would learn — with no further access needed — that this specific tenant has a Fair Housing complaint or privileged legal correspondence on file. That is exactly the confirmation-of-a-sensitive-fact risk the scope document's own Section 7, Question 5 names: *"some signal that something exists but is restricted... risks confirming to someone that a sensitive thread exists at all."* A genuinely indistinguishable "no results" — identical, byte for byte, to a real true non-match — is the only default that doesn't leak that fact, and it's what this design already produces with zero special-case code, purely as a consequence of Finding 1's structural gate.

## Finding 8 — The HELD-bucket output for Peter/DO/counsel

**Mason's Condition 4, built as a simple, real export — not a search feature into the held bucket itself.** GET /api/archive-search/held-review-export (admin-only): every row where screening\_result = 'held', returning missive\_conversation\_id, delivered\_at (earliest message in the thread), subject (may legitimately be empty — see the same owner-email-completeness caveat complaint-tracking-technical-spec.md's Design Decision 3 already documents for its own Layer 0 match), from\_address, which mechanism tripped the hold (domain match, keyword-phrase match, or the new co-occurrence match — never the literal matched term or phrase itself, same restraint as Finding 1), and a Missive deep link (missive\_conversation\_id → the real conversation, same "link back to Missive" pattern the product doc's own search results already use). Exported as CSV, same mechanism reused from Finding 5 (one simple exported list, not two different one-off review tools) — Peter, the DO, or counsel opens it, reviews each item once, closes the loop by hand (outside this system — this route does not build a disposition/closure workflow, matching Mason's own instruction that this is "not a search feature into the held bucket itself").

archive\_search.held\_review\_export\_generated (audit\_log, human, low, processing) logs each time the export is pulled — details: { row\_count }.

**A real, honest note on overlap with** complaints.held\_legal\_fair\_housing, **not glossed over:** complaint-tracking's own pipeline runs checkThread( ) independently against whatever mail it processes, and creates its own held placeholder rows in complaints when it finds one. This tool's screening pass also runs checkThread( ), independently, against the (mostly non-overlapping, since complaint-tracking has not been pointed at the historical backlog) set of rows it screens. **These two "held" surfaces are not reconciled or deduplicated against each other.** For the 254,056-message historical backlog specifically, this tool's held-review export is the first and, until someone builds a reconciliation step, the only authoritative record of what the retroactive pass found — named here as a real, accepted, not-yet-solved overlap, not a conflict (both surfaces are independently correct about the same underlying fact), tracked as an Open Item.

## Finding 9 — The retention/deletion-clock consequence, stated plainly

The original migration's own words, quoted directly, are the governing language here: *"RULE 1 — WHILE pipeline\_status = 'pending': no row may be deleted for age or any other retention reason... RULE 2 — ONCE pipeline\_status = 'processed' for a given row... AND that row's filter outcome (held / tagged / tier) is available to check... Tier 2 HELD -> LEGAL HOLD. No deletion clock at all... Everything else... -> 4 years from delivered\_at, then eligible for deletion."*

**This is the real, understood consequence of this spec, stated plainly, not left implicit:** once this screening pass runs against the 254,056-message backlog, and (per "Resolving Finding 2") also flips pipeline\_status to 'processed' for every row it screens, two things become true simultaneously, for the first time, at once, across a quarter-million rows:

1. **The RULE 2 PREREQUISITE the original migration named as missing is satisfied** — screening\_result is exactly the "new column on this table (a held/tier flag)" that note said would need to exist before any age-based deletion job could validly be built. This is a genuine, positive step toward the retention policy the migration already committed to — not a new commitment being made here.

2. **The 4-year deletion clock (California Code of Civil Procedure Section 337, per the migration's own reasoning) starts running, retroactively from each message's real delivered\_at date, on every non-held row, the moment this pass completes.** This means both 'clear' and 'flagged\_protected\_class' rows — **stated explicitly, not left implicit: a Fair-Housing-flagged row is not a legal hold.** The migration's own RULE 2 language is a strict binary — Tier 2 HELD, or "everything else" — and this codebase's own established precedent (complaints.flagged\_protected\_class's real column comment: "advisory tag only, never a hold") already settles which side of that binary a content-check flag falls on. So a 'flagged\_protected\_class' row is excluded from search (Finding 1/7) but is **not** shielded from the retention clock the way a 'held' row is — it ages out on the same 4-year schedule as ordinary mail. A real share of the 254,056 messages (anything delivered more than 4 years before the pass runs, across both of those outcomes) would become **immediately eligible** for deletion under RULE 2, the instant a deletion job is ever built, with no further waiting period. **No deletion job exists yet** — nothing is deleted by this spec or this pass — but the clock itself starts for real, not hypothetically, the moment pipeline\_status flips. Only 'held' rows get the opposite: a permanent legal hold, no clock at all, retained until an attorney affirmatively releases it.

Peter should understand this as a real, dated, structural step, not routine bookkeeping — flagged explicitly in "Open Items" for confirmation before the batch pass runs, exactly per this document's own repeated discipline of naming a real consequence rather than letting it arrive as a surprise later.

## The Search Mechanism — A Real Recommendation

**Recommendation: Postgres full-text search via tsvector + a GIN index, not ILIKE.** At 254,056 rows and growing, an ILIKE '%term%' scan has no usable index (a leading wildcard defeats a plain B-tree, and this codebase has no pg\_trgm extension enabled anywhere today) — every search would be a full table scan of two large text columns. A tsvector + GIN index is the standard, proven mechanism for exactly this shape of query (word/phrase search across a large text corpus) and is what the product doc's own framing ("like a search engine, but for Rincon's own email history") actually describes.

```
```sql
ALTER TABLE missive_message_intake
ADD COLUMN search_document tsvector
GENERATED ALWAYS AS (
to_tsvector('english', coalesce(subject, '') || ' ' || coalesce(body_text, ''))
) STORED;

CREATE INDEX idx_missive_message_intake_search_document
ON missive_message_intake USING GIN (search_document);
```
```

subject + body\_text only — deliberately **not** from\_address. An email address tokenizes oddly under the 'english' search configuration and would dilute relevance for the actual use case (searching what was *said*, not who sent it). Sender lookup is served instead by a plain, optional ?from= query parameter (ILIKE against from\_address on the already-narrowed, already-screened row set — cheap at this scale once the tsvector predicate has already done the real filtering, no dedicated index needed for it).

**Ordering:** delivered\_at DESC, **exactly per the product doc** — "newest first," **not** relevance-ranked. No ts\_rank needed for v1.

**Snippet generation: computed in application code, not ts\_headline().** For the bounded result-page size this tool returns (tens of rows, not thousands), fetching body\_text for the matched page and finding a ~200-character window around the first matched term in JS is simpler than wiring a Postgres-side ts\_headline() call through Supabase's query builder (which has no clean pass-through for it without a dedicated RPC function) — consistent with CLAUDE.md's "simple is better than clever." ts\_headline() is the real, available upgrade path if app-code snippeting proves inadequate later; not built for v1.

**A real, confirmed technical fact, not assumed:** a query against missive\_message\_intake\_search\_safe (the view) still uses the GIN index on the base table, because a plain, non-security-barrier view is flattened into the outer query by Postgres's planner — the WHERE screening\_result = 'clear' predicate and the search\_document @@ websearch\_to\_tsquery(...) predicate are evaluated together against the real table, not against a separately materialized view result. No second index is needed on the view itself (views cannot carry their own indexes in Postgres regardless).

```
```js
const { data } = await supabase
  .from('missive_message_intake_search_safe')
  .select('id, mailbox_key, missive_conversation_id, subject, from_address, delivered_at, body_text')
  .textSearch('search_document', query, { type: 'websearch', config: 'english' })
  .order('delivered_at', { ascending: false })
  .limit(RESULT_LIMIT);
```
```

**Known, accepted limitation, stated plainly:** tsvector with English stemming/stop-words is not built for partial-word, unit-number, or misspelling-tolerant matching (e.g., a specific apartment number embedded in an address). Named as a real gap, not solved here — a pg\_trgm trigram index is the natural v2 answer if this proves to matter in practice; not built for v1, per the same "don't build more than asked" discipline CLAUDE.md states directly.

## The Screening Pass — Concretely, Against Real Tables

New module, `archive-search/lib/screening-pass.js`, called only from `router.js`'s `internalRouter` (`POST /api/archive-search/process-pending`, `x-cron-secret-gated`, **no automatic schedule**, matching every other batch pipeline's established posture in this codebase — Design Decision 16's precedent, `email-intake/router.js`'s own manually-triggered-first pattern, and `complaint-tracking`'s identical posture for its own pipeline).

1. Query `missive_message_intake` WHERE `screening_result` IS NULL, ordered by `delivered_at` ASC, **in resumable chunks** (a LIMIT/cursor, not one unbounded query) — see "Build Size and Runtime" for why an unattended run against tens of thousands of conversations is itself an operational risk worth designing against, not a detail to leave to Q.

2. Group by `missive_conversation_id`. Adapt each group into `checkThread()`'s expected shape using `toThreadShape()` — **reused directly from** `complaint-tracking/lib/thread-adapter.js`, **not re-derived** (it is already a real, exported, tested function; requiring it cross-directory is an established pattern in this codebase, e.g. `process-pending-messages.js` already requires `../../email-intake/lib/privilege-filter` and `../../owner-tenant-notes/router`).

3. Run `checkThread()` (`email-intake/lib/privilege-filter.js`) — identical function, identical call, to `complaint-tracking`'s own Step 2.

4. **Held:** write `screening_result = 'held'`, `screening_tags = null`, `screening_version`, `screening_completed_at = NOW()`, `pipeline_status = 'processed'` on every message row in the conversation. Write `archive_search.screening_held` (system, high, collection, details: { `source_missive_conversation_id`, `message_count`, `hold_mechanism` } — never the matched term). No content check, no self-report call, ever, for a held thread — identical restraint to `complaint-tracking`'s own Step 3.

5. **Not held:** build `threadText` via `threadFullText()` (also reused from `thread-adapter.js`). Call `selfReportFairHousingContent()` (Finding 2), then `checkClaim()` with that self-report. Write `screening_result = 'flagged_protected_class'` or `'clear'`, `screening_category` (if flagged), `screening_tags` (any Tier 1 TAG label from `checkThread()`), `screening_version`, `screening_completed_at = NOW()`, `pipeline_status = 'processed'` on every message row in the conversation.

6. If flagged: write `archive_search.screening_flagged_protected_class` (actor\_type: 'system' if Tier A keyword-only, 'ai\_agent' otherwise — same exact discipline `complaint-tracking`'s own equivalent event already uses — high, processing, details: { `flagged_category`, `matched_layer`, `terms_version`, `self_report_version` } — never the matched term or the self-report's own free text).

7. A conversation that throws mid-processing is left with `screening_result` still NULL (never marked 'clear' on a failure path) so the next run retries it — identical safety discipline to `complaint-tracking`'s own per-conversation error handling.

8. One summary event per whole run, not one per row (matching `email_intake.missive_sync_run`'s own established "one row per batch, aggregate counts" convention, not `complaint_tracking.created`'s "one row per new record" convention — there is no new domain record being created here for a 'clear' outcome, so a 'clear'-only row-level event would be pure noise at this volume):  
`archive_search.batch_pass_run` (system, low, collection, details: { `conversations_processed`, `held`, `flagged`, `clear`, `errors`, `chunk_start`, `chunk_end` }).

**Versioning (screening\_version):** a single compact string identifying the full combination in effect for a given row — e.g. 'archive-search-screening-v1', bumped whenever `privilege-keywords.js`'s `TERMS_VERSION`, `protected-class-terms.js`'s `TERMS_VERSION`, or `fair-housing-batch-self-report.js`'s `FAIR_HOUSING_SELF_REPORT_VERSION` changes materially — satisfying GOVERNANCE.md Rule 5's "every decision must reference the version in effect" without a second config table for what is really a code-version question, same reasoning `complaint_tracking.extracted_by`'s own design already used.

## Proposed Data Model (for Neo to finalize)

```
```sql
-- =====
-- ALTER: missive_message_intake — new screening columns (Finding 1)
-- =====

ALTER TABLE missive_message_intake
ADD COLUMN screening_result TEXT,
ADD COLUMN screening_category TEXT,
ADD COLUMN screening_tags JSONB,
ADD COLUMN screening_version TEXT,
ADD COLUMN screening_completed_at TIMESTAMPTZ;

ALTER TABLE missive_message_intake
ADD CONSTRAINT missive_message_intake_screening_result_check
CHECK (screening_result IS NULL OR screening_result IN ('held', 'flagged_protected_class', 'clear'));

ALTER TABLE missive_message_intake
ADD CONSTRAINT missive_message_intake_screening_category_required
CHECK (screening_result IS DISTINCT FROM 'flagged_protected_class' OR screening_category IS NOT NULL);

-- Full-text search document — subject + body_text only, see "The Search
-- Mechanism" for why from_address is deliberately excluded.
--
-- OPERATIONAL NOTE (Neo's review, fix 3) — read before running this
-- against the real table. Unlike the five plain columns added above
-- (metadata-only in Postgres 11+), a GENERATED ... STORED column
```

-- requires Postgres to compute and write the expression for every
-- EXISTING row — a full rewrite of missive_message_intake, all
-- 254,056+ rows, under an exclusive lock, while the live Missive sync
-- cron job (which writes to this exact table on its own schedule)
-- keeps running. This project has no staging copy of the data to
-- rehearse this against (same standing caveat 20260905020000's own
-- migration-gate self-check already carries) — so the real table is
-- the only place this gets tested. Peter/Q should expect a real pause
-- on this one ALTER TABLE statement specifically (duration scales with
-- table + body_html/body_text size, not measured here) and should not
-- run it during a Missive sync job's active window without accounting
-- for that.

```
ALTER TABLE missive_message_intake
ADD COLUMN search_document tsvector
GENERATED ALWAYS AS (
to_tsvector('english', coalesce(subject, '') || ' ' || coalesce(body_text, ''))
) STORED;
```

-- CONCURRENTLY (Neo's review, fix 3): a plain CREATE INDEX takes a lock
-- that blocks writes to missive_message_intake for the build duration —
-- real risk at 254,056+ rows with the live sync cron writing
-- concurrently. CONCURRENTLY avoids that lock but CANNOT run inside a
-- transaction block — each statement below must be run on its own in
-- Supabase's SQL Editor, not pasted together with the rest of this
-- migration as one script, or Postgres will reject it.

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_missive_message_intake_search_document
ON missive_message_intake USING GIN (search_document);
```

-- Archive search's own work queue — mirrors idx_missive_message_intake_pending's
-- partial-index-on-status convention exactly, but keyed to this tool's own
-- screening state, deliberately independent of pipeline_status.

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_missive_message_intake_screening_pending
```

ON missive_message_intake (delivered_at)

WHERE screening_result IS NULL;

CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_missive_message_intake_delivered_at

ON missive_message_intake (delivered_at);

-- =====

-- VIEW: missive_message_intake_search_safe (Finding 1) — mirrors

-- maintenance_claims_decision_safe's single-table WHERE, no join, plus

-- security_barrier (Neo's review, fix 1 — see "Resolving Finding 1"

-- for why this view needs it and maintenance_claims_decision_safe

-- doesn't).

-- =====

CREATE OR REPLACE VIEW missive_message_intake_search_safe

WITH (security_barrier = true) AS

SELECT *

FROM missive_message_intake

WHERE screening_result = 'clear';

-- =====

-- team_member_tool_roles — 'archive_search' (12th tool value),

-- 'searcher' (10th role value). Same DROP-then-ADD pattern this

-- constraint has used repeatedly. Current lists confirmed directly

-- against 20260910000000_complaint_tracking_schema.sql (tool) and

-- 20260902020000_add_maintenance_coordinator_role.sql (role) — the real,

-- current state, not assumed.

-- =====

ALTER TABLE team_member_tool_roles

DROP CONSTRAINT IF EXISTS team_member_tool_roles_tool_check;

ALTER TABLE team_member_tool_roles

ADD CONSTRAINT team_member_tool_roles_tool_check

CHECK (tool IN (

'insurance_compliance', 'maintenance_history', 'security_deposit', 'call_stats',

```
'content_engine', 'leadsimple_application_screening', 'leadsimple_delinquency',  
'leadsimple_operations', 'approval_briefing', 'owner_tenant_notes',  
'complaint_tracking', 'archive_search'  
));
```

```
ALTER TABLE team_member_tool_roles  
  
DROP CONSTRAINT IF EXISTS team_member_tool_roles_role_check;  
  
ALTER TABLE team_member_tool_roles  
  
ADD CONSTRAINT team_member_tool_roles_role_check  
  
CHECK (role IN (  
  
'admin', 'director_of_operations', 'property_manager', 'inspection_coordinator',  
  
'pod_lead', 'reviewer', 'contributor', 'leasing_reviewer', 'maintenance_coordinator',  
  
'searcher'  
));
```

```
-- No seed/grant INSERT — who holds 'searcher' vs 'admin' for  
  
-- tool='archive_search' is Peter's call, made only after the validation  
  
-- sample (Finding 5) has passed, per the scope document's own still-open  
  
-- sequencing note (Section 4).  
  
...
```

audit_log — no schema change. Every event named above (archive_search.screening_held, .screening_flagged_protected_class, .batch_pass_run, .query_performed, .message_opened, .validation_sample_reviewed, .held_review_export_generated) uses actor_type IN ('human', 'ai_agent', 'system'), privacy_category IN ('collection', 'processing'), risk_level IN ('low', 'medium', 'high') — all already legal under audit_log's real, current CHECK constraints (20260815000000_audit_log_rule1_compliance.sql), confirmed directly.

Data Inventory (GOVERNANCE.md Rule 4)

This spec adds no new PII to missive_message_intake — its Rule 4 inventory (20260905020000) already covers body_html/body_text/subject/from_address/to_addresses/cc_addresses/bcc_addresses in full. What changes:

- agents_with_access: was NONE (that migration's own words: "no AI agent, LLM call, or extraction step reads this table"). **This is no longer true after this build** — this is the real, load-bearing change this spec makes to that inventory: the screening pass's new fair-housing-batch-self-report.js classifier

(Claude, via ANTHROPIC_API_KEY, same already-resolved DPA/Commercial-Terms answer this exact use already has from operational_notes-SPEC.md Section 8) now reads full, unscreened body_text content for non-held threads. Also: Hub users holding 'searcher' or 'admin' for tool='archive_search' — but **only ever against** missive_message_intake_search_safe, never the base table (search routes; the screening pass itself, which necessarily reads the base table, is the one narrow exception, per Finding 1).

- privacy_category: unchanged ('collection') for the base table; the new columns are a 'processing' artifact of that same already-collected content, not a new collection event.
- retention_policy: unchanged in substance — the RULE 1/RULE 2 policy already set by Mason (2026-09-05) is not being rewritten here. What changes is that RULE 2's own PREREQUISITE is, for the first time, satisfiable — see "Resolving Finding 9."
- ccpa_exportable / ccpa_deletable: unchanged (TRUE/TRUE, same targeted-redaction convention). The new screening_result/screening_category/screening_tags/screening_version/screening_completed_at/search_document columns are **not** included in the redaction target list — search_document is a GENERATED column and is automatically recomputed to reflect a redacted subject/body_text the moment those are redacted (no separate redaction step needed for it), and screening_result itself is exactly the kind of audit-continuity metadata this schema's existing redaction convention already preserves on purpose (the fact "this row was screened, and found X" stays; the correspondence itself does not).

Access / Roles in the Hub

team_member_tool_roles.tool gains 'archive_search'; role gains 'searcher' (see "Proposed Data Model"). Two roles checked for this tool: 'searcher' (search + open a message) and 'admin' (search + open a message + the validation-sample export + the held-review export + the search-activity log). 'admin' is a superset of 'searcher', not a separate track.

```
```js
```

```
const ARCHIVE_SEARCH_SEARCH_ROLES = ['searcher', 'admin'];
```

```
const ARCHIVE_SEARCH_ADMIN_ROLES = ['admin'];
```

```
```
```

attachArchiveSearchRole / requireArchiveSearchAccess / requireArchiveSearchAdmin — the same three-function shape (attach.../require...Access/require...Role) every Hub tool implements independently, evaluated only against rows where tool='archive_search' — never inherited from what 'admin' means on any other tool, the same explicit-allow-list discipline owner-tenant-notes/router.js already documents as a hard-won lesson.

No seed/grant row. Per Finding 5's sequencing answer: Peter grants 'admin' first (to himself and whoever does the validation-sample review), and only grants 'searcher' to the remaining 6 of the 8 confirmed people (Stephen, Dio, Leo, Regina, Marci, Elizabeth) and to Caylee (the named, accepted exception) once that sample has passed with zero confirmed misses.

Routes Needed (for Q)

- GET /api/archive-search?q=...&from=... — the search itself (requireArchiveSearchAccess). Logs archive_search.query_performed.
- GET /api/archive-search/message/:id — full message view, filtered to missive_message_intake_search_safe by id (requireArchiveSearchAccess). Logs archive_search.message_opened.
- POST /api/archive-search/process-pending — the screening pass (internal router, x-cron-secret-gated, manually triggered only).
- GET /api/archive-search/validation-sample-export — Finding 5 (requireArchiveSearchAdmin).
- GET /api/archive-search/held-review-export — Finding 8 (requireArchiveSearchAdmin).
- GET /api/archive-search/auth/me — role/name lookup for the dashboard, same pattern every other tool's equivalent route already uses.

Build Size and Runtime — A Real Consideration, Not a Detail Left to Q

254,056 messages is not 254,056 screening-pass units of work — it's grouped by conversation first, and the real conversation count is unknown until Q actually runs the Step-1 grouping query (this spec does not guess a number it hasn't measured). What **is** known: every non-held conversation costs one narrow AI call (Finding 2) on top of the free, deterministic hold check — a real, bounded, but non-trivial one-time cost in both API spend and wall-clock time, not a detail to discover mid-run.

Concretely, this spec asks Q to build the screening pass to run in resumable chunks from the start (a LIMIT, not an unbounded single query — see "The Screening Pass," step 1) and recommends a first, count-only dry run (SELECT missive_conversation_id, COUNT(*) FROM missive_message_intake WHERE screening_result IS NULL GROUP BY 1 — no AI calls, no writes) before committing to a live run, so the real scope is measured, not assumed, before any cost is spent. A conversation that fails mid-run is retried automatically on the next chunk (per "The Screening Pass," step 7) — an unattended, single, hours-long process against tens of thousands of live API calls is itself an operational risk this design is built to avoid by construction, not something Q should discover the hard way.

Open Items — Needs Confirming Before This Gets Built

1. **The retention/deletion-clock consequence (Finding 9)** — flipping pipeline_status to 'processed' on 254,000+ rows starts a real 4-year deletion-eligibility clock, retroactively from each message's own delivered_at, on ordinary correspondence **and** on 'flagged_protected_class' rows alike — a Fair-Housing flag is excluded from search but is not a legal hold and is not shielded from this clock; only

'held' rows are. No deletion job exists yet, nothing is deleted by this build — but the clock itself becomes real, immediately, for a real share of the archive. Peter should confirm this explicitly before the batch pass runs.

2. The permanent effect on complaint-tracking's own historical reach (Finding 2) — once this pass runs, complaint-tracking's WHERE pipeline_status = 'pending' ingestion query will never again see the pre-batch-pass historical backlog, meaning it will never categorize any of the 254,056 archived messages into its own complaints table, unless a human deliberately resets specific rows back to 'pending' later. This is the intended resolution of Section 3's sequencing risk — but it is permanent by default, and Peter should confirm he's not expecting complaint-tracking to eventually catch up on old mail some other way.

3. The dual "held" surfaces (Finding 8) — this tool's own screening_result = 'held' rows and complaint-tracking's own complaints.held_legal_fair_housing = TRUE rows are two independent, unreconciled records of the same underlying fact for any conversation both pipelines happen to touch. Not a conflict, but not unified either — a real Open Item, not solved here.

4. The real conversation count and the real per-call cost — unmeasured until Q runs the Step-1 dry-run query described in "Build Size and Runtime." Recommend Q reports this back to Peter before the live batch pass is scheduled, not after it's already run.

5. Whether screening_tags (Tier 1 TAG labels, e.g. regulatory_matter) should be surfaced in the v1 search UI at all — this spec stores them (cheap, harmless, and useful for the held-review export's own "which mechanism fired" column) but does not require Tron to display them; a product-level call, not an engineering one.

Before Any of This Runs For Real

Per GOVERNANCE.md and CLAUDE.md: this document is a draft technical spec. Before Neo applies any migration or Q writes any code against real data, this spec needs Peter's approval. Before the screening pass ever runs against the real 254,056-message archive, compliance/archive-search-ai-risk-assessment.md (Finding 3) needs to exist and be reviewed — it is written and delivered alongside this document, see below. Before any of the 8 confirmed people get 'searcher' access, the Finding 5 validation sample needs to come back with zero confirmed misses. No step in that sequence is implied by this document being written — each is a real, separate gate.